

Post-Quantum OpenID Connect with KEMTLS-Inspired Key Establishment: Prototype and Evaluation

Aniket Gupta (IIT2023018)

Indian Institute of Information Technology, Allahabad

Repository: <https://github.com/42-answer/PQC>

Abstract—As quantum computing compromises classical cryptographic primitives, securing authentication infrastructure has become a critical objective. Standard Post-Quantum TLS (PQTLS) introduces severe performance overhead due to the massive size of post-quantum digital signatures. To address this, the presented Python prototype implements a KEMTLS-inspired key-establishment protocol, completely substituting digital signatures with highly efficient Key Encapsulation Mechanisms (KEMs) for both confidentiality and authentication. An OpenID Connect (OIDC) authorization pipeline is deployed over this transport layer, facilitating the issuance and verification of post-quantum signed JSON Web Tokens (PQ-JWTs). The end-to-end flow is accessible via a web interface and JSON APIs.

Performance characteristics of the post-quantum primitives are evaluated using an integrated microbenchmarking suite. The reported quantitative metrics reflect in-process execution times and cryptographic payload sizes, ensuring deterministic reproducibility. The underlying system architecture is mapped, deviations from standard compliance are identified, and a roadmap toward production-ready network integration is outlined.

Index Terms—post-quantum cryptography, Kyber (ML-KEM), ML-DSA, Falcon, KEMTLS, OpenID Connect, JWT, benchmarking.

I. SCOPE AND REPRODUCIBILITY

A. Introduction

The transition to post-quantum cryptography is an urgent requirement for securing modern authentication systems against future quantum threats. The standard TLS 1.3 protocol achieves key establishment via ephemeral Diffie-Hellman and relies on digital signatures for server authentication. A straightforward approach to post-quantum security is Post-Quantum TLS (PQTLS), which operates as a direct drop-in replacement, substituting classical algorithms with post-quantum equivalents. However, post-quantum digital signatures (such as ML-DSA) present a significant drawback: their public keys and signatures are substantially larger than traditional algorithms like RSA or ECDSA. As illustrated in Figure 1, PQTLS maintains the identical structural rounds and message flow of TLS 1.3 but inherits severe communication and computational overhead due to the massive size of these post-quantum signatures [1].

To mitigate this overhead, we adopt an alternative protocol design called KEMTLS [2], [1]. Because post-quantum Key Encapsulation Mechanisms (KEMs) are fundamentally more

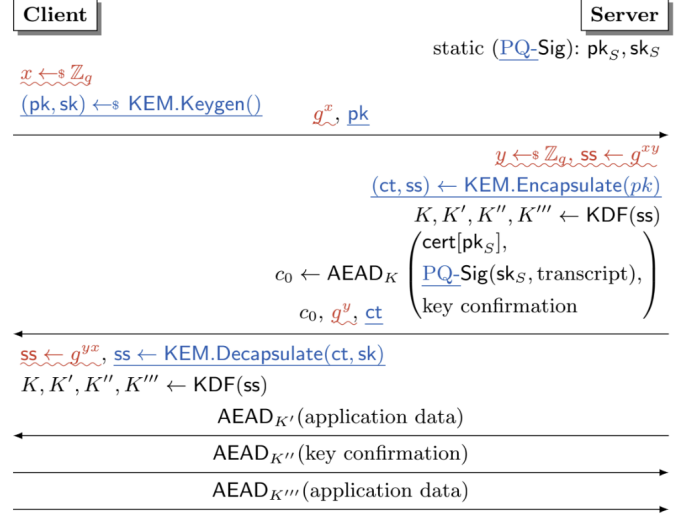


Fig. 1. Comparison between TLS 1.3 handshake (black and red) and PQTLS (black and blue). Note that both protocols require the same number of rounds and messages [1].

efficient and require smaller cryptographic material than post-quantum signatures, KEMTLS eliminates handshake signatures entirely. Instead, it leverages KEMs for both confidentiality and authentication. As depicted in Figure 2, KEMTLS restructures the handshake flow: authentication is achieved by encapsulating a shared secret against the server's long-term post-quantum KEM public key, which is advertised in its certificate. Only the legitimate server possessing the corresponding private key can decapsulate this secret, authenticating the connection while circumventing the bandwidth penalties of post-quantum digital signatures.

This optimized transport foundation supports an application-layer OpenID Connect (OIDC) authorization-code flow [3], [4], [5]. The architecture is validated through an integrated microbenchmarking suite and a web-based interface. All quantitative results analyzed in this report are extracted directly from the generated artifacts to guarantee deterministic reproducibility.

The primary technical contributions include:

- **KEMTLS validation:** An implementation demonstrating the mitigation of post-quantum signature overhead via KEM-based authentication.

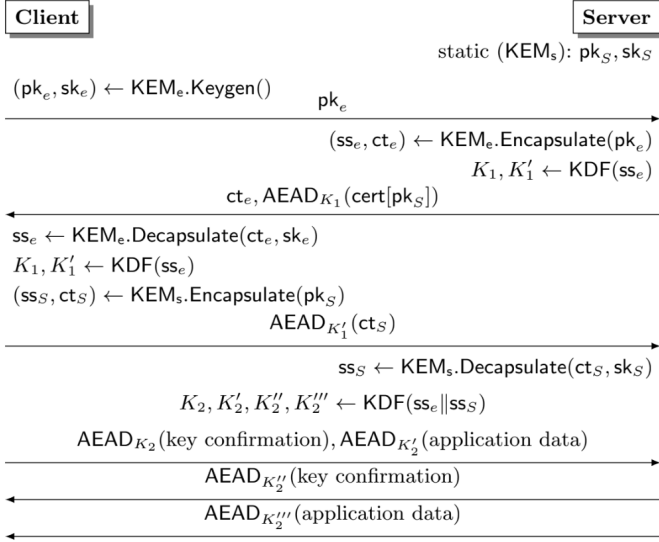


Fig. 2. Overview of KEMTLS with server-only authentication [1].

- **Primitive integration:** A post-quantum wrapper layer exposing Kyber KEM and signature operations for modular composition.
- **OIDC pipeline:** An authorization demonstrator generating and verifying post-quantum signed ID tokens.
- **Reproducible methodology:** An artifact-driven reporting structure that programmatically translates benchmarking data into academic visualizations.

Project repository: <https://github.com/42-answer/PQC>

The quantitative analysis relies exclusively on checked-in benchmark datasets (CSV/JSON).

B. How to Run (README-aligned summary)

The README documents a full setup using liboqs plus a Python virtual environment (Python 3.8+; tested on Linux/macOS). At a high level:

- Install liboqs and ensure the shared library is discoverable (e.g., via `LD_LIBRARY_PATH`).
- Activate the project environment via `source setup_env.sh`.
- Run the UI demo with `python ui/app.py` and optionally validate endpoints with `python test_ui_endpoints.py`.

C. Standards Context and Related Work

The prototype is framed using established standards and prior work, without proposing new protocol specifications. The identity flow follows the structure of the OAuth 2.0 authorization-code exchange [3] and references the OIDC Core profile [4]; tokens use JWT/JWS container formats [6], [7]. For key establishment, the prototype adopts the central KEMTLS idea of removing handshake signatures [2]. The underlying KEM and signature algorithms align with NIST standardization outcomes and references [8], [9], [10]. The implementation uses Open Quantum Safe tooling (liboqs and

its Python bindings) to access PQ algorithms in a reproducible software stack [11].

See detailed instructions in the GitHub repository README: <https://github.com/42-answer/PQC>

D. Code Layout (as used by this report)

The README organizes the implementation into the following components. This report uses the top-level layout (`src/`, `ui/`, `docs/`, `examples/`, `benchmark_results/`, `report/`). A nested `PQC/` directory also exists with a similar structure; this report references the top-level paths.

- `src/pq_crypto/`: wrappers for PQ KEM and signature operations.
- `src/kemtls/`: KEMTLS protocol structures and client/server modules.
- `src/oidc/`: PQ-signed JWT handling and an in-memory OIDC server/client.
- `src/benchmarks/run_benchmarks.py`: benchmark execution logic.
- `src/benchmarks/generate_pdf_report.py`: benchmark-report generation used by the benchmarking subsystem.
- `src/docs/generate_technical_doc.py`: technical documentation generator.
- `benchmark_results/benchmark_results.csv` and `benchmark_results/benchmark_results.json`: stored benchmark results used in all tables/figures.
- `ui/app.py`: Flask UI demo and API endpoints.
- `ui/templates/`: HTML templates for the demo pages.
- `test_ui_endpoints.py`: smoke tests against the running UI.
- `setup_env.sh`: environment activation (venv + liboqs paths), as documented in the README.
- `examples/`: example scripts for running demo flows.
- `docs/`: architecture and quickstart documents.
- `report/`: this LaTeX report plus the asset generator used to produce figures and tables.

E. Software Stack and Execution Environment

The prototype is implemented in Python and is intended to run inside a virtual environment created by `setup\_env.sh`. The setup script configures `PYTHONPATH` for in-repository imports and extends `LD_LIBRARY_PATH` so the system liboqs shared library can be located at runtime. The Flask UI additionally sets `OQS_INSTALL_DIR=/usr/local` to prefer an existing liboqs installation.

The primary technologies and libraries used for the implementation and evaluation are:

- **Post-quantum primitives:** liboqs via `liboqs-python` (`oqs` module) for Kyber KEM and PQ signature algorithms [11].

- **Web demonstration:** Flask (and Flask-CORS) for the UI and JSON API endpoints (`ui/app.py`).
- **Benchmarking harness:** an in-process microbenchmark runner built around `time.perf_counter()` (`src/benchmarks/run/_benchmarks.py`).
- **Plotting and report assets:** NumPy and Matplotlib for generating figures and LaTeX table fragments from stored artifacts (`report/generate/_assets.py`).
- **Endpoint validation:** the requests HTTP client for smoke-testing UI endpoints (`test/_ui/_endpoints.py`).
- **Build toolchain:** L^AT_EX (IEEEtran class, TikZ, and listings) for PDF generation from `report/main.tex`.

The repository also declares additional Python dependencies (e.g., PyJWT, Authlib, cryptography, python-dotenv, and PyYAML) that are common in OAuth/JWT ecosystems. In this prototype, the PQ-JWT handler implements JWT serialization explicitly to keep the token format and signature inputs easy to inspect.

F. Reproduction Pipeline

The following pipeline reproduces the report assets (tables/figures) from stored benchmark artifacts. It does not re-run cryptographic benchmarks; it re-plots and re-tabulates the existing results.

```
# from repo root
source setup_env.sh
python3 report/generate_assets.py
# build the PDF (pdflatex must be installed)
cd report
pdflatex -interaction=nonstopmode main.tex
pdflatex -interaction=nonstopmode main.tex
```

Listing 1. Re-creating figures and tables used by this report.

II. OPENID CONNECT ARCHITECTURE

Securing the transport layer must be paired with robust application-layer identity workflows. OpenID Connect 1.0 [4] establishes an identity layer atop the OAuth 2.0 framework [3]. While OAuth 2.0 governs profile authorization, OpenID Connect introduces definitive identity management, enabling a Relying Party (RP) to cryptographically verify user authenticity via an OpenID Provider (OP) [5].

As outlined by Li and Mitchell [5], the OpenID Connect ecosystem relies on interactions between four core entities:

- 1) **End User (U):** The person attempting to access protected resources.
- 2) **User Agent (UA):** Typically a web browser routing requests and responses.
- 3) **OpenID Provider (OP):** The server (e.g., Google or our Flask implementation) that authenticates the user and generates identity assertions.
- 4) **Relying Party (RP):** The application providing the on-line service, which consumes the OP’s identity assertions to grant access.

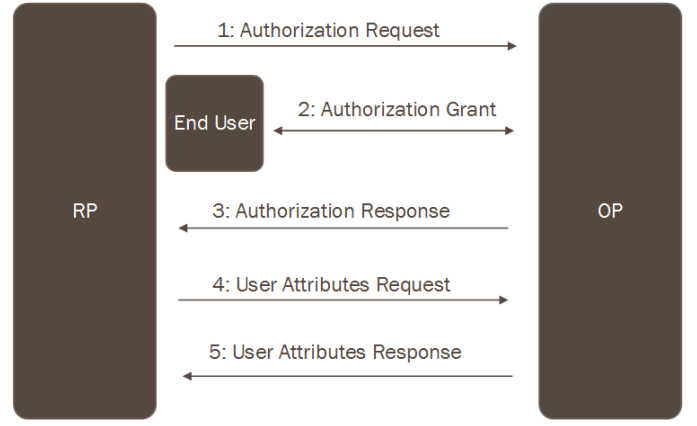


Fig. 3. Interactions within the OpenID Connect framework during authentication. Our prototype secures the generated identity tokens using post-quantum signatures [5].

The implemented prototype executes an OIDC authorization-code sequence where the OpenID Provider issues JSON Web Tokens (JWTs) as identity assertions. Standard RSA/ECDSA signing algorithms are substituted with post-quantum equivalents, producing PQ-signed JWTs that conform to the existing OpenID Connect specification.

III. SYSTEM ARCHITECTURE

A. Layered Design

The architectural design is strictly layered: post-quantum primitives form the base, followed by a KEMTLS key-establishment protocol [2], and culminating in the OIDC application flow. Figure 4 details this structural hierarchy in relation to the codebase modules.

This stack operates in two modes. Functionally, the web interface triggers in-process endpoints executing the OAuth 2.0 and OIDC authorization sequence [3], [4]. This mechanism handles the issuance and verification of ID tokens within standard JWT/JWS containers [6], [7]. Analytically, the benchmark module evaluates primitive operations and end-to-end performance. The “KEMTLS handshake” metric specifically measures a simulated key establishment (encapsulation, decapsulation, and HKDF key derivation [12]), representing the cryptographic processing overhead independent of network latency [13].

B. Implementation Mapping to Repository Modules

To keep the architecture verifiable, we map each conceptual layer to the specific source files that implement it and note how those files are exercised. This mapping also makes the evaluation path explicit: the Flask UI drives the demo endpoints, the benchmark runner defines the timed regions and writes CSV/JSON artifacts, and the report pipeline consumes those artifacts to produce plots and tables. Primary implementation paths used by the demo and benchmark suite:

- **Layer 1 (primitives):** `src/pq_crypto/kem.py`, `src/pq_crypto/signature.py`, `src/pq_crypto/utils.py`.

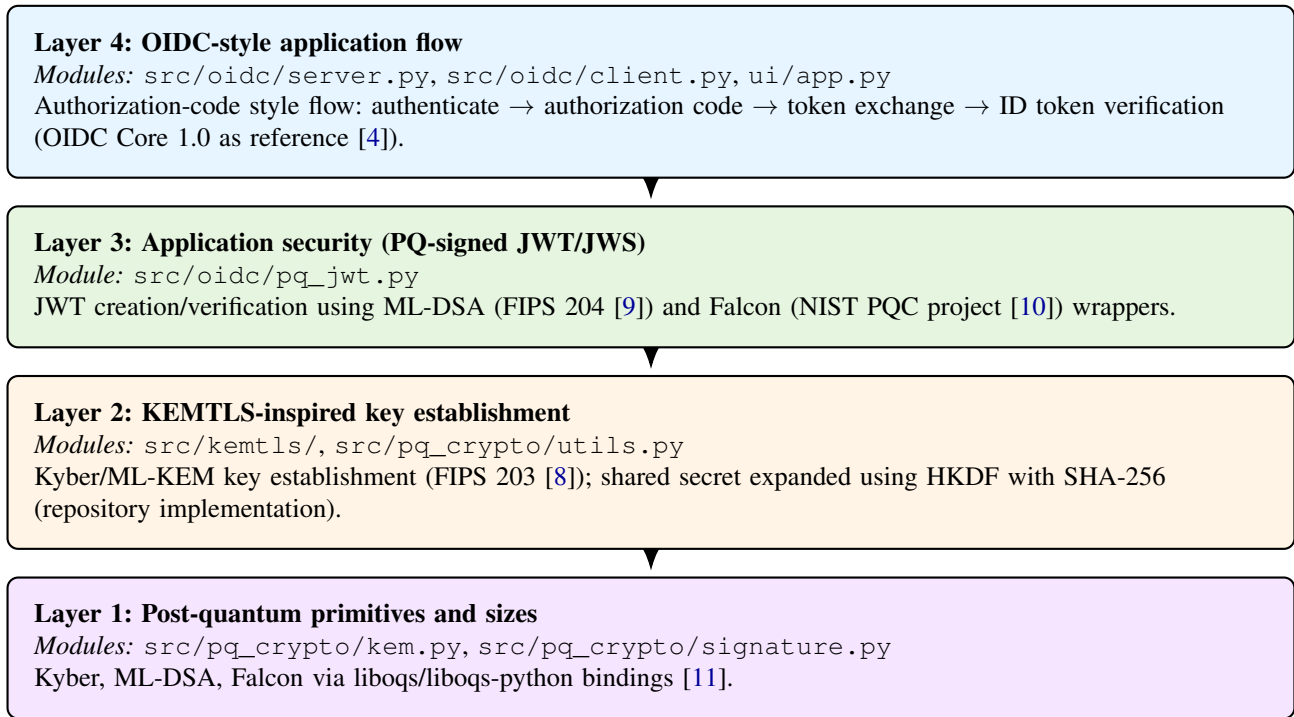


Fig. 4. Layered architecture diagram aligned with the repository implementation.

- **Layer 2 (KEMTLS-style):** `src/kemtls/protocol.py` plus `src/kemtls/client.py` and `src/kemtls/server.py`.
- **Layer 3 (PQ-JWT):** `src/oidc/pq_jwt.py`.
- **Layer 4 (OIDC-style demo):** `src/oidc/server.py`, `src/oidc/client.py`, and the UI wiring in `ui/app.py`.
- **Benchmarks and artifacts:** `src/benchmarks/run_benchmarks.py`, stored results under `benchmark_results/`, and report-asset generation via `report/generate_assets.py`.

Implementation-level interaction (how the layers compose in code):

- **Primitive wrappers and KDF utilities.** The PQ wrappers in `src/pq_crypto/` provide a thin integration layer over `liboqs` (via `liboqs-python`) for KEM and signature operations. The same package also contains utility code (e.g., HKDF-style expansion using SHA-256 in repository code) that is reused by higher layers.
- **KEMTLS-inspired key establishment.** The modules under `src/kemtls/` define message structures and client/server roles suitable for a KEMTLS-inspired handshake. For evaluation, the stored “Full KEMTLS Handshake” row corresponds to a benchmark-suite simulation built from the same underlying KEM operations and key derivation logic, executed in-process.
- **PQ-signed JWT handling.** The token logic in `src/oidc/pq_jwt.py` constructs and verifies JWT/JWS-style containers while delegating signing and verification to the PQ signature wrappers.

- **OIDC-style flow orchestration.** The in-memory OIDC server/client modules under `src/oidc/` implement the authorization-code style sequencing used by the demo (authorization, code issuance, token exchange, ID token verification). The Flask UI endpoints in `ui/app.py` call into these modules to execute the flow as a single in-process sequence.

Methodology hook (where measurements attach): the benchmark runner in `src/benchmarks/run_benchmarks.py` defines the measurement boundaries for each artifact row. Primitive-level rows wrap individual operations (key generation, encapsulation/decapsulation, signing/verification). Protocol-level rows wrap complete in-process sequences: the KEMTLS-style key-establishment simulation and the end-to-end OIDC-style flow. These timings are written to CSV/JSON under `benchmark_results/` and are subsequently transformed into figures and tables by `report/generate_assets.py`.

C. Implementation Status and Coverage

The repository contains both core implementations and prototype wiring:

- **Implemented primitives:** Kyber KEM operations and PQ signatures (ML-DSA variants, Falcon variants) are wrapped and benchmarked.
- **KEMTLS components:** protocol structures and client/server modules exist; the benchmarked “Full KEMTLS Handshake” in stored results corresponds to a key-establishment simulation implemented in the benchmark suite (KEM encaps/decaps + key derivation).

- **OIDC demo flow:** the Flask UI endpoint `/api/oidc/flow` executes an in-process authorization-code style sequence via server methods and PQ-signed ID token verification. The final “UserInfo” step is simplified to a direct user lookup (not a full bearer-token validated UserInfo implementation).

D. Implementation-Centric Architecture Views

Beyond the conceptual layering, Figures 5 and 6 illustrate how the repository modules connect in practice.

IV. DEMO AND EVALUATION PIPELINES

A. Flask Demo UI Pipeline

The UI exposes routes for demonstrations and JSON APIs. The smoke-test script `test_ui_endpoints.py` checks that the following APIs return structured results when the UI is running:

- `POST /api/kemtls/handshake`: Kyber keygen + encaps + decaps and shared-secret equality check.
- `POST /api/signatures/test`: keygen + sign + verify; includes a negative test.
- `POST /api/jwt/create`: create and verify a PQ-signed JWT; returns token size.
- `POST /api/oidc/flow`: runs a five-step demo flow (authentication, code generation, token exchange, ID token verification, simplified userinfo retrieval).

Separately, the UI also exposes `GET /api/benchmarks` to return the stored benchmark JSON used for visualization (this endpoint is used by the UI pages but is not exercised by `test_ui_endpoints.py`).

B. UI Pipeline Diagram

The project includes a lightweight Flask-based UI that provides interactive pages and JSON API endpoints for demonstrating the prototype’s post-quantum components. The UI supports a KEMTLS-style handshake demonstration (Kyber encaps/decaps and shared-secret check), PQ signature testing, PQ-signed JWT creation/verification, and an OIDC-style authorization-code flow.

The following diagram is a compact, implementation-aligned view of how the UI is executed and validated.

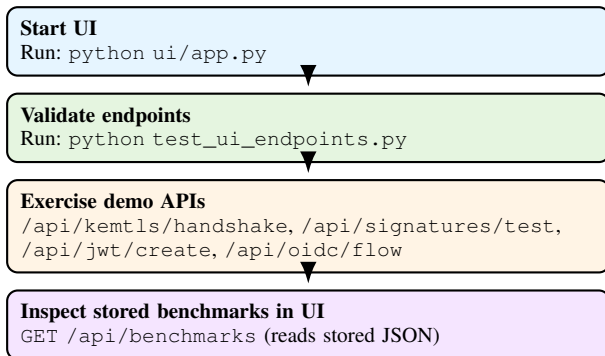


Fig. 7. UI pipeline with implementation-aligned steps.

C. Benchmark → Report Pipeline

This diagram summarizes how benchmark execution produces stored artifacts, which are then transformed into report-ready figures and tables.

D. Benchmarking Pipeline

The benchmark suite in `src/benchmarks/run_benchmarks.py` measures:

- 1) Kyber KEM keygen/encapsulation/decapsulation (three Kyber variants),
- 2) PQ signature keygen/sign/verify (ML-DSA and Falcon variants),
- 3) JWT creation and verification (subset of signature algorithms),
- 4) a “Full KEMTLS Handshake” simulation (key establishment + HKDF-based derivation),
- 5) an “End-to-End OIDC Flow” benchmark (in-process demo sequence).

The stored results contain 32 benchmark rows (see Appendix A).

V. BENCHMARKING AND RESULTS (STORED ARTIFACTS)

A. Dataset and Provenance

All benchmark figures and tables in Section 4 are derived from the checked-in artifacts: `benchmark_results/benchmark_results.csv` and `benchmark_results/benchmark_results.json`. The plotting and tabulation script `report/generate_assets.py` is data-driven and produces the included PNG plots and LaTeX table fragments.

B. Metrics Summary (from stored artifacts)

The stored dataset contains **32** benchmark rows spanning **10** distinct operations and **10** configurations/algorithm labels. Key dataset-level metrics (microbenchmarks, in-process):

- **Protocol-level timings (mean ms):** Full KEMTLS Handshake ≈ 0.0409 ms; End-to-End OIDC Flow ≈ 0.2049 ms.
- **Kyber KEM operation means (ms):** observed range across Kyber512/768/1024 keygen/encap/decap is approximately 0.0129 to 0.1003 ms in the stored artifacts.
- **JWT sizes (bytes):** ID token sizes produced by *JWT Creation* range from **1193** to **4734** bytes across the benchmarked algorithms (verification rows record size as 0 in the artifacts).
- **Signature sizes (bytes):** signature sizes recorded for signing range from **656** to **4627** bytes in the stored artifacts.

C. KEM Operation Timings (Kyber)

This subsection summarizes key generation, encapsulation, and decapsulation timings across Kyber variants in the stored dataset.

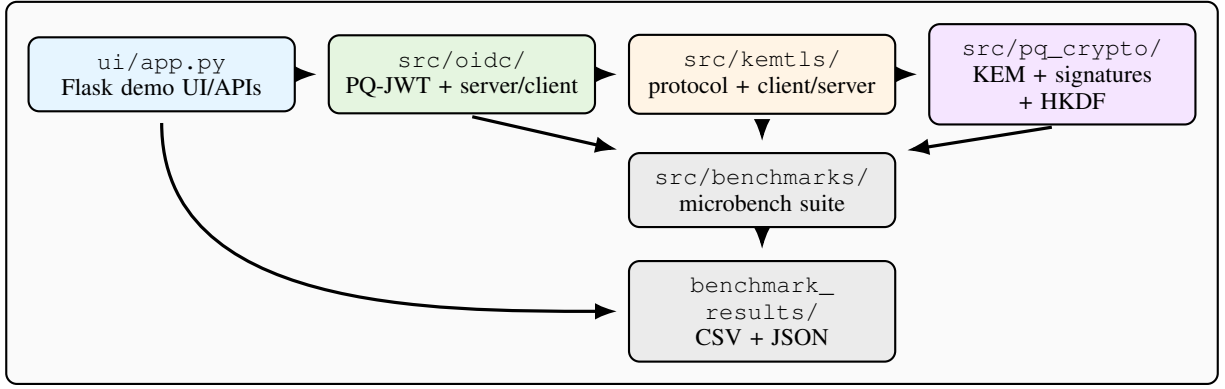


Fig. 5. Module-level architecture (arrows denote calls/reads in the repository; the benchmark suite writes stored artifacts).

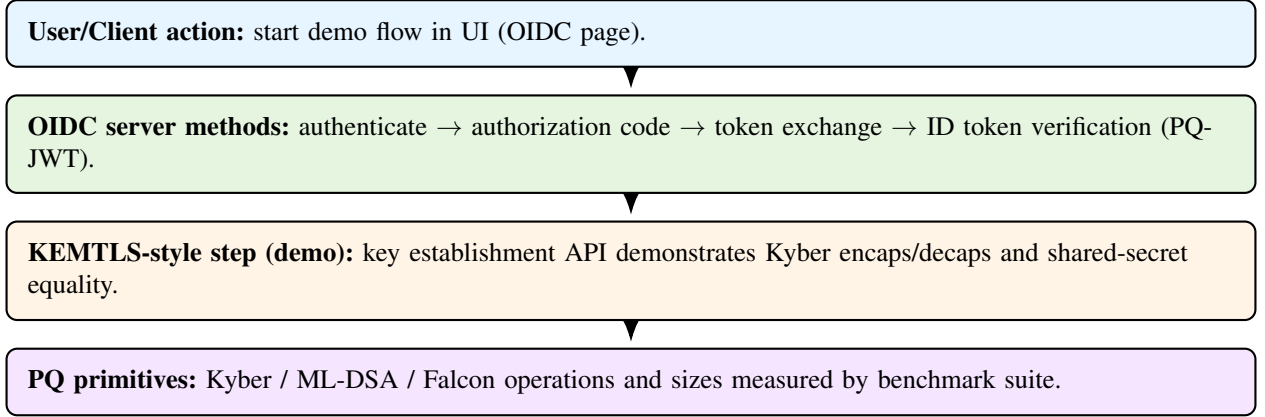


Fig. 6. End-to-end architecture view of the prototype pipeline (implementation-aligned).

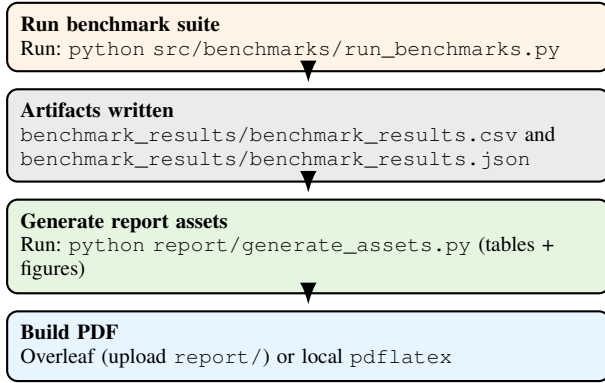


Fig. 8. Benchmark-to-report pipeline from execution to PDF generation.

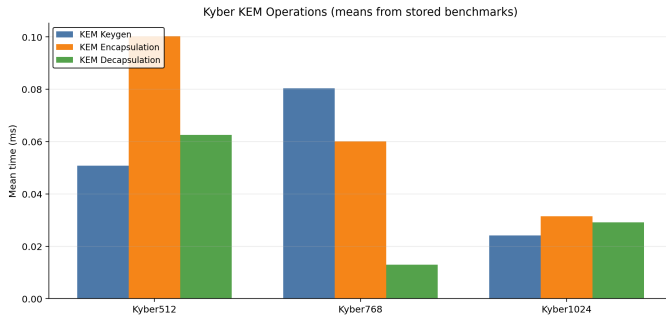


Fig. 9. Kyber KEM operation means (ms) from stored results.

D. Signature Timings (ML-DSA and Falcon)

This subsection reports signing and verification timings across ML-DSA and Falcon variants as benchmarked in-process.

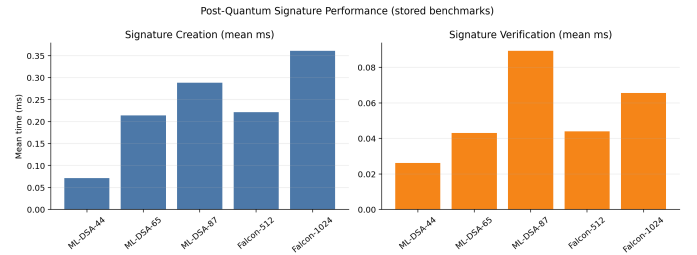


Fig. 10. Signature creation and verification means (ms) from stored results.

E. JWT Performance and Token Sizes

This subsection reports JWT creation/verification time and the resulting ID token sizes for the PQ-signed JWT implementation.

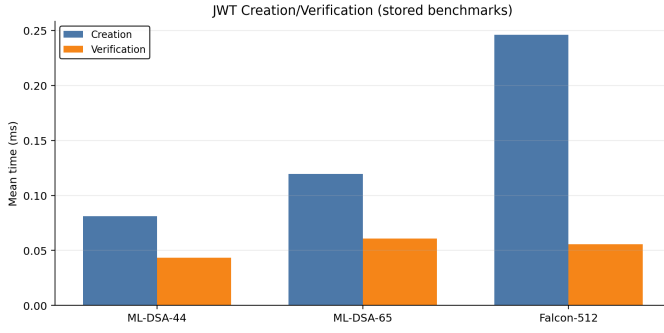


Fig. 11. JWT creation vs. verification time (means, stored benchmarks).

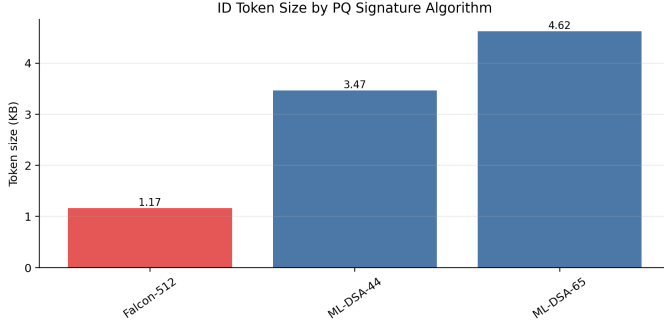


Fig. 12. ID token size by PQ signature algorithm (stored benchmarks).

F. Key Tables and Protocol Summary

The following tables are generated from the same stored artifacts and provide the protocol-level summary rows plus JWT size/time measurements.

TABLE I
PROTOCOL-LEVEL BENCHMARK SUMMARY.

Operation	Configuration	Mean (ms)	Median (ms)	Stdev (ms)	Iters
End-to-End OIDC Flow	Complete Authorization Code Flow	0.2049	0.1729	0.0859	20
Full KEMTLS Handshake	Kyber512 + ML-DSA-44	0.0409	0.0387	0.0129	50

TABLE II
JWT CREATION PERFORMANCE AND TOKEN SIZES.

Algorithm	Mean (ms)	Median (ms)	Stdev (ms)	Iters	Size (B)
Falcon-512	0.2461	0.2550	0.0277	100	1193
ML-DSA-44	0.0812	0.0687	0.0342	100	3549
ML-DSA-65	0.1197	0.1074	0.0605	100	4734

G. Additional Visualizations

These additional visualizations use the stored mean and standard-deviation fields and highlight variability and size-vs-time tradeoffs.

VI. METHODOLOGY

A. Benchmark Data Provenance

All numeric results in this report are taken from the checked-in benchmark artifacts: `benchmark_results/benchmark_results.csv` and `benchmark_results/benchmark_results.json`. The plotting and tabulation code in `report/generate_assets.py` reads these inputs and produces:

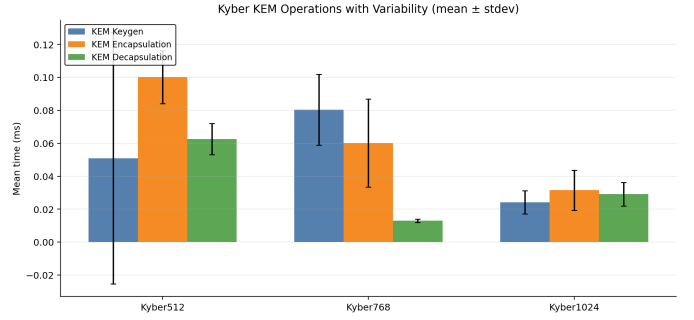


Fig. 13. KEM operations with variability (mean $\pm$ stdev), derived from stored results.

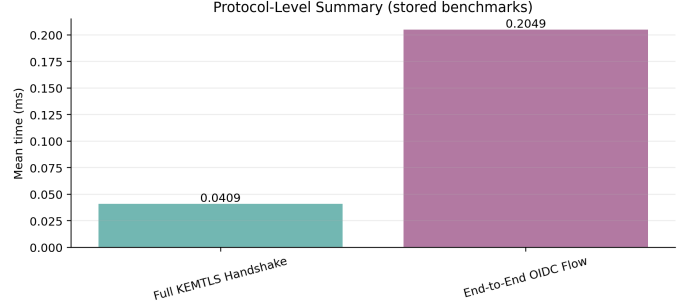


Fig. 14. Protocol-level benchmark summary (mean ms), derived from stored results.

- figures under `report/figures/`, and
- LaTeX table fragments under `report/tables/`.

B. Execution Model

The benchmark runner (`src/benchmarks/run_benchmarks.py`) is an in-process microbenchmark suite for cryptographic operations and for prototype flows. The harness uses `time.perf_counter()` around the measured operation and aggregates mean/median/stdev/min/max over a fixed number of iterations per operation (recorded in the corresponding artifact row).

The OIDC-style flow in the repository follows the structure of OAuth 2.0 authorization code exchange [3] and uses JWT/JWS token containers [6], [7]. Key expansion in the KEMTLS-style workflow is consistent with HKDF usage [12]; however, the measured protocol-level rows remain in-process benchmarks and should not be interpreted as networked TLS 1.3 latency [13].

C. What “End-to-End” Means Here

The “End-to-End OIDC Flow” item in the stored results corresponds to a complete authorization-code style sequence executed within one Python process. It includes token issuance and ID token verification using the PQ-JWT handler. In the Flask UI (`ui/app.py`), the `UserInfo` step is intentionally simplified (direct user lookup) rather than a fully standards-conformant `UserInfo` endpoint that validates bearer access tokens.

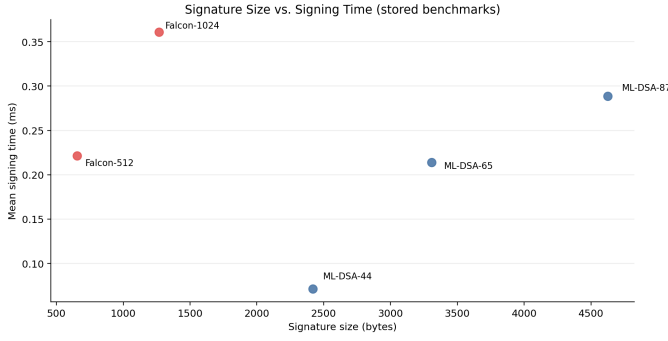


Fig. 15. Signature size vs. signing time (mean ms), derived from stored results.

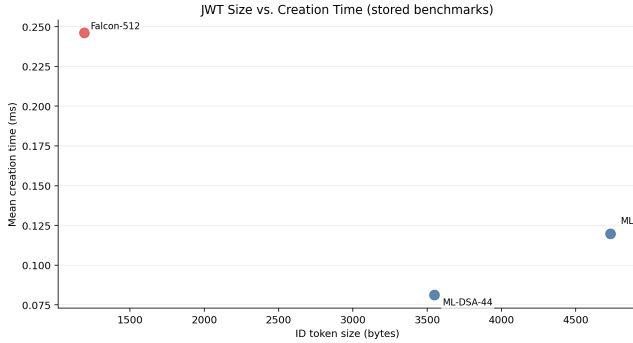


Fig. 16. JWT ID token size vs. creation time (mean ms), derived from stored results.

VII. ALGORITHMS AND PARAMETERS

A. KEM: Kyber Variants

The benchmark dataset includes Kyber512, Kyber768, and Kyber1024 for key generation, encapsulation, and decapsulation. These correspond to the Kyber/ML-KEM family standardized as ML-KEM in FIPS 203 [8]. The stored artifacts also record encapsulation ciphertext sizes (bytes) for each parameter set.

B. Signatures: ML-DSA and Falcon

The dataset includes ML-DSA-44/65/87 (FIPS 204 [9]) and Falcon-512/1024 (NIST PQC project [10]) across key generation, signing, and verification. The stored artifacts include signature sizes (bytes) for signing operations.

C. JWT: PQ-Signed ID Tokens

JWT creation and verification are benchmarked for ML-DSA-44, ML-DSA-65, and Falcon-512. The stored artifacts include the ID token sizes (bytes) produced by the corresponding implementation.

VIII. SCOPE AND LIMITATIONS

The implementation is a prototype and microbenchmark suite. The following scope constraints are important for interpretation:

- **Microbenchmarks:** all timings are in-process measurements and should not be interpreted as network latency.

- **KEMTLS handshake metric:** “Full KEMTLS Handshake” is a repository-implemented simulation (Kyber operations plus key derivation), not a socket-level handshake between KEMTLSClient and KEMTLSServer.
- **UserInfo in the UI demo:** the UI uses direct user lookup for the final step; the demo UserInfo handler returns `invalid_token` and the client-side method is not implemented.

IX. DEVELOPMENT STATUS, NOVELTY, AND FUTURE WORK

In its current form, the codebase serves as an end-to-end demonstrator for PQ-protected identity and token issuance. It integrates (i) PQ primitives, (ii) a KEMTLS-inspired key-establishment workflow [2], and (iii) an OIDC-style authorization-code flow exposed through repeatable UI and API endpoints.

A. What is Novel in This Repository

The contribution is primarily *integration and reproducibility*: one codebase connects PQ KEM and signature primitives with an identity-token flow, and the report is generated from checked-in benchmark artifacts through a deterministic pipeline. This keeps the results auditable and the demonstration repeatable.

B. Future Scope Toward Deployment

To move from a demonstrator to a deployable system, the following work is needed:

- **Standards-aligned OIDC behavior:** complete token validation for UserInfo, align endpoint behavior with OIDC Core, and add security checks (nonce/state handling, error semantics, token lifetime) consistent with OAuth/OIDC security guidance [4], [3], [14], [15], [16].
- **Networked transport integration:** connect the OIDC endpoints to a real KEMTLS-enabled transport (client/server over sockets) and measure networked latency under concurrency.
- **Key management and hardening:** secure key storage, rotation, and audit logging; isolate cryptographic operations; add threat-model-driven checks.
- **Operational readiness:** configuration management, CI for tests/benchmarks, and packaging/deployment patterns suitable for production environments.
- **Interoperability and hybrid modes:** evaluate PQ-hybrid handshakes and token signing options for incremental deployment and compatibility.
- **Security analysis and robustness:** add a clear threat model, formalize trust boundaries (UI/server/crypto), and perform negative testing and fuzzing for token parsing and protocol state.

C. Roadmap Direction (Brief)

A practical sequence of next steps toward a stronger research artifact and, later, a deployment-oriented implementation is:

- **Near-term (weeks):** tighten protocol semantics and error handling against OAuth/OIDC requirements [3], [4]; implement bearer-token validation for UserInfo [15]; add PKCE support for the authorization-code flow [16]; expand negative tests.
- **Mid-term (1–2 months):** integrate a networked client/server transport for the KEMTLS-inspired layer; measure latency under concurrency; add key storage and rotation interfaces; document an explicit threat model with testable assumptions [14].
- **Long-term (semester scale):** evaluate hybrid deployment options (classical+PQ), interoperability constraints, and cross-implementation portability; extend benchmarks to include network effects and larger payloads; pursue formal validation of token parsing and state machines.

X. CONCLUSION

This prototype validates the structural integration of post-quantum cryptography within established authentication protocols. The implementation maps post-quantum primitives to a KEMTLS key-establishment simulation, mitigating the latency penalties associated with massive digital signatures. The resulting transport layer supports an operational OIDC authorization flow reliant on PQ-signed JSON Web Tokens.

Methodological reproducibility is enforced across the evaluation pipeline:

- **Deterministic Evaluation:** All visual and numeric artifacts are programmatically derived from static, checked-in benchmarking datasets (CSV/JSON).
- **Structural Mapping:** Abstract protocol layers are explicitly coupled to distinct codebase modules (primitives, KEMTLS, and PQ-JWT) for audibility.
- **Empirical Scope:** The benchmarking suite tracks 32 execution and size metrics, isolating the isolated cryptographic overhead of the KEMTLS handshake and the integrated OIDC sequence.

Future efforts must transition the system from an in-process demonstrator to a networked implementation. Critical next steps involve deploying a fully compliant UserInfo endpoint, establishing a socket-bound KEMTLS transport layer, and evaluating throughput constraints under realistic network concurrency.

REFERENCES

- [1] S. Celi, A. Faz-Hernández, N. Sullivan, G. Tamvada, L. Valenta, T. Wiggers, B. Westerbaan, and C. A. Wood, “Implementing kemtls,” in *Progress in Cryptology–LATINCRYPT 2021*, 2021.
- [2] P. Schwabe, D. Stebila, and T. Wiggers, “Kemtls: Eliminating handshake signatures in tls 1.3,” IACR ePrint 2020/534, 2020. [Online]. Available: <https://eprint.iacr.org/2020/534>
- [3] D. Hardt, “The oauth 2.0 authorization framework,” 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6749>
- [4] OpenID Foundation, “Openid connect core 1.0,” 2014. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html
- [5] W. Li and C. J. Mitchell, “Analysing the security of google’s implementation of openid connect,” *arXiv preprint arXiv:1508.01707*, 2015.
- [6] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [7] —, “Json web signature (jws),” 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7515>
- [8] National Institute of Standards and Technology, “Fips 203: Module-lattice-based key-encapsulation mechanism standard (ml-kem),” 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [9] —, “Fips 204: Module-lattice-based digital signature standard (ml-dsa),” 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [10] —, “Post-quantum cryptography project.” [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [11] Open Quantum Safe, “liboqs and liboqs-python.” [Online]. Available: <https://github.com/open-quantum-safe/liboqs>
- [12] H. Krawczyk and P. Eronen, “Hmac-based extract-and-expand key derivation function (hkdf),” 2010. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5869>
- [13] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [14] D. Lodderstedt, M. McGloin, and P. Hunt, “OAuth 2.0 threat model and security considerations,” 2013. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6819>
- [15] M. Jones and D. Hardt, “The oauth 2.0 authorization framework: Bearer token usage,” 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6750>
- [16] J. Sakimura, N. Bradley, and M. Jones, “Proof key for code exchange by oauth public clients (pkce),” 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7636>

APPENDIX A COMPLETE BENCHMARK DATASET

TABLE III: Complete benchmark dataset (derived from stored benchmark artifacts).

Operation	Algorithm	Mean (ms)	Median (ms)	Stdev (ms)	Min (ms)	Max (ms)	Iters	Size (B)
End-to-End Flow	OIDC Complete Authorization Code Flow	0.2049	0.1729	0.0859	0.1399	0.4612	20	0
Full KEMTLS Handshake	Kyber512 + ML-DSA-44	0.0409	0.0387	0.0129	0.0381	0.1296	50	0
JWT Creation	Falcon-512	0.2461	0.2550	0.0277	0.1854	0.3207	100	1193
JWT Creation	ML-DSA-44	0.0812	0.0687	0.0342	0.0465	0.1939	100	3549
JWT Creation	ML-DSA-65	0.1197	0.1074	0.0605	0.0649	0.3922	100	4734
JWT Verification	Falcon-512	0.0557	0.0468	0.0167	0.0457	0.1767	100	0
JWT Verification	ML-DSA-44	0.0435	0.0426	0.0064	0.0417	0.1051	100	0
JWT Verification	ML-DSA-65	0.0607	0.0597	0.0041	0.0587	0.0904	100	0
KEM Decapsulation	Kyber1024	0.0291	0.0269	0.0071	0.0207	0.0460	100	0
KEM Decapsulation	Kyber512	0.0625	0.0592	0.0095	0.0577	0.1456	100	0
KEM Decapsulation	Kyber768	0.0129	0.0127	0.0010	0.0125	0.0199	100	0
KEM Encapsulation	Kyber1024	0.0315	0.0240	0.0122	0.0228	0.0663	100	1568
KEM Encapsulation	Kyber512	0.1003	0.0994	0.0161	0.0763	0.2290	100	768
KEM Encapsulation	Kyber768	0.0601	0.0736	0.0267	0.0248	0.1289	100	1088
KEM Keygen	Kyber1024	0.0241	0.0220	0.0070	0.0182	0.0587	100	0
KEM Keygen	Kyber512	0.0508	0.0494	0.0763	0.0162	0.7798	100	0
KEM Keygen	Kyber768	0.0804	0.0765	0.0216	0.0707	0.2817	100	0
Sign	Falcon-1024	0.3607	0.3561	0.0161	0.3466	0.4443	100	1269
Sign	Falcon-512	0.2212	0.2399	0.0318	0.1772	0.2858	100	656
Sign	ML-DSA-44	0.0711	0.0571	0.0362	0.0354	0.1982	100	2420
Sign	ML-DSA-65	0.2138	0.1730	0.1301	0.0523	0.6883	100	3309
Sign	ML-DSA-87	0.2884	0.2275	0.1641	0.0791	0.7371	100	4627
Signature Keygen	Falcon-1024	15.8008	14.5125	3.5410	12.7385	28.5174	100	0
Signature Keygen	Falcon-512	5.3442	4.8952	1.5479	3.9154	12.6677	100	0
Signature Keygen	ML-DSA-44	0.0325	0.0284	0.0209	0.0273	0.2341	100	0
Signature Keygen	ML-DSA-65	0.0500	0.0424	0.0133	0.0418	0.1249	100	0
Signature Keygen	ML-DSA-87	0.0683	0.0655	0.0118	0.0643	0.1769	100	0
Verify	Falcon-1024	0.0655	0.0640	0.0060	0.0632	0.1167	100	0
Verify	Falcon-512	0.0440	0.0499	0.0112	0.0336	0.1121	100	0
Verify	ML-DSA-44	0.0262	0.0256	0.0028	0.0251	0.0524	100	0
Verify	ML-DSA-65	0.0431	0.0418	0.0035	0.0405	0.0572	100	0
Verify	ML-DSA-87	0.0893	0.0648	0.0454	0.0629	0.1870	100	0

APPENDIX B RAW CSV SNAPSHOT (FOR AUDIT)

Listing 2. benchmark_results.csv (snapshot conv packaged with this report)

```

Operation,Algorithm,Mean (ms),Median (ms),Stdev (ms),Min (ms),Max (ms),Iterations,Size (bytes)
"KEM Keygen","Kyber512",0.0508,0.0494,0.0763,0.0162,0.7798,100,0
"KEM Encapsulation","Kyber512",0.1003,0.0994,0.0161,0.0763,0.2290,100,768
"KEM Decapsulation","Kyber512",0.0625,0.0592,0.0095,0.0577,0.1456,100,0
"KEM Keygen","Kyber768",0.0804,0.0765,0.0216,0.0707,0.2817,100,0
"KEM Encapsulation","Kyber768",0.0601,0.0736,0.0267,0.0248,0.1289,100,1088
"KEM Decapsulation","Kyber768",0.0129,0.0127,0.0010,0.0125,0.0199,100,0
"KEM Keygen","Kyber1024",0.0241,0.0220,0.0070,0.0182,0.0587,100,0
"KEM Encapsulation","Kyber1024",0.0315,0.0240,0.0122,0.0228,0.0663,100,1568
"KEM Decapsulation","Kyber1024",0.0291,0.0269,0.0071,0.0207,0.0460,100,0
"Signature Keygen","ML-DSA-44",0.0325,0.0284,0.0209,0.0273,0.2341,100,0
"Sign","ML-DSA-44",0.0711,0.0571,0.0362,0.0354,0.1982,100,2420
"Verify","ML-DSA-44",0.0262,0.0256,0.0028,0.0251,0.0524,100,0
"Signature Keygen","ML-DSA-65",0.0500,0.0424,0.0133,0.0418,0.1249,100,0
"Sign","ML-DSA-65",0.2138,0.1730,0.1301,0.0523,0.6883,100,3309
"Verify","ML-DSA-65",0.0431,0.0418,0.0035,0.0405,0.0572,100,0
"Signature Keygen","ML-DSA-87",0.0683,0.0655,0.0118,0.0643,0.1769,100,0

```

```

"Sign", "ML-DSA-87", 0.2884, 0.2275, 0.1641, 0.0791, 0.7371, 100, 4627
"Verify", "ML-DSA-87", 0.0893, 0.0648, 0.0454, 0.0629, 0.1870, 100, 0
"Signature Keygen", "Falcon-512", 5.3442, 4.8952, 1.5479, 3.9154, 12.6677, 100, 0
"Sign", "Falcon-512", 0.2212, 0.2399, 0.0318, 0.1772, 0.2858, 100, 656
"Verify", "Falcon-512", 0.0440, 0.0499, 0.0112, 0.0336, 0.1121, 100, 0
"Signature Keygen", "Falcon-1024", 15.8008, 14.5125, 3.5410, 12.7385, 28.5174, 100, 0
"Sign", "Falcon-1024", 0.3607, 0.3561, 0.0161, 0.3466, 0.4443, 100, 1269
"Verify", "Falcon-1024", 0.0655, 0.0640, 0.0060, 0.0632, 0.1167, 100, 0
"Full KEMTLS Handshake", "Kyber512 + ML-DSA-44", 0.0409, 0.0387, 0.0129, 0.0381, 0.1296, 50, 0
"JWT Creation", "ML-DSA-44", 0.0812, 0.0687, 0.0342, 0.0465, 0.1939, 100, 3549
"JWT Verification", "ML-DSA-44", 0.0435, 0.0426, 0.0064, 0.0417, 0.1051, 100, 0
"JWT Creation", "ML-DSA-65", 0.1197, 0.1074, 0.0605, 0.0649, 0.3922, 100, 4734
"JWT Verification", "ML-DSA-65", 0.0607, 0.0597, 0.0041, 0.0587, 0.0904, 100, 0
"JWT Creation", "Falcon-512", 0.2461, 0.2550, 0.0277, 0.1854, 0.3207, 100, 1193
"JWT Verification", "Falcon-512", 0.0557, 0.0468, 0.0167, 0.0457, 0.1767, 100, 0
"End-to-End OIDC Flow", "Complete Authorization Code Flow
", 0.2049, 0.1729, 0.0859, 0.1399, 0.4612, 20, 0

```

APPENDIX C

PUBLIC REPOSITORY FOR SUPPLEMENTARY MATERIALS

A public Google Drive folder is provided to support peer review and reproducibility. It contains:

- Detailed project documentation
- Demonstration video of the system
- Benchmark results in .json, .xls, and .pdf formats
- Generated graphs and performance analysis plots
- Additional artifacts and supporting information

Available at: <https://drive.google.com/drive/folders/10vI3nyteF3sNEliO031Q8r9rMdjAiOOY>